

Mastr — Practical Execution Plan

A parallel, contracts-first work split for a 3-person team

Team AB-05

- Kaveen Kawsal S — CB.AI.U4AID23119
- Sri Somesh S — CB.AI.U4AID23141
- Sai Chakrith — CB.AI.U4AID23143

Supervisor: Dr. Kritesh Kumar Gupta | Domain: Gen AI, NLP

This plan replaces the strictly sequential split shown in the review presentation with a **contracts-first, parallel** structure — all three members start building in Week 1, against shared data contracts instead of against each other's finished work.

Why we're splitting it this way

The presentation's split (Diagnostic → Graph → Tutoring → Dashboard) has hidden serial dependencies. If followed literally, the dashboard can't show anything until the backend works, and the tutoring module can't be tested until gaps are actually detected. Two of the three people would sit idle in Week 1.

The fix: agree on the exact shape of the data passed between modules (a **contract**) before writing real logic. Everyone builds against dummy data matching that shape, then we swap dummy data for real modules two at a time during scheduled integration checkpoints. Nobody blocks anybody.

The six contracts

Lock these as JSON shapes today, in a shared doc or a schemas.py file in the repo. Everything else in this plan builds against them.

Contract	Shape (fields)	Owner
Question	id, sub_skill, difficulty (1-5), text, options, correct_answer	Sai Chakrith
SubSkillGraph	nodes: [...], edges: [{from, to}]	Sri Somesh
MasteryEstimate	learner_id, sub_skill, score (0-1), confidence, n_attempted	Kaveen
Gap	learner_id, sub_skill, score, priority_rank	Kaveen
TutoringContent	gap_id, sub_skill, explanation, source_chunks, retest_question_ids	Sri Somesh
LearnerState	learner_id, stage, history: [...]	Sai Chakrith

Integration checkpoints

Every ~2 weeks, one mock gets replaced by the real module. This keeps the whole pipeline runnable end-to-end at all times, even while individual pieces are still unfinished.

Checkpoint	Week	What gets swapped
1	~Wk 3	Kaveen's dummy 5-node graph → Sri Somesh's real prerequisite graph
2	~Wk 5	Sai Chakrith's mock /diagnostic endpoint → Kaveen's real staircase engine
3	~Wk 8	Mock /tutor endpoint → Sri Somesh's real RAG pipeline
4	~Wk 10	Wire the full retest-to-verify loop end-to-end, remove all mocks

Track 1 — Diagnostic Engine & Mastery Scoring

Owner: Kaveen Kawsal S

What this track delivers

- An adaptive staircase test that converges on a per-sub-skill mastery score in a bounded number of questions
- A scoring model that turns raw responses into the MasteryEstimate and Gap contracts
- The convergence study (Expected Outcome #3 from the review deck)

How to build it — suggested approach

Step 1 (Week 1, no dependencies): Build a tiny fake question bank yourself — 15-20 made-up questions across 3-4 sub-skills, each tagged with difficulty 1-5. Don't wait for the real question bank; you only need something structurally correct to test the algorithm logic.

Step 2 — Start rule-based, not full IRT: A true Item Response Theory model is more math than you need for an MVP. Start with a simple staircase rule: begin at difficulty 3, answer correct → next question one level harder, answer wrong → one level easier, stop after a fixed number of questions per sub-skill (e.g. 6) or once 2 consecutive questions don't change the estimate. This alone satisfies Objective #2. Upgrade to a 1-parameter Rasch/IRT model later if time in Sem 2 allows — it's a drop-in replacement, not a rewrite.

Step 3 — Scoring: Compute mastery as a weighted average of correctness by difficulty (harder correct answers count more), or a simple logistic regression with difficulty as the single feature. Bayesian updating is a nice-to-have upgrade, not a Week 1 requirement.

Step 4 — Convergence study: Once the algorithm works, simulate 100+ fake learners with known 'true' ability, run the staircase on each, and plot how many questions it takes for the estimate to stabilize within a tolerance. This becomes a chart in your final report for free.

Suggested tools

- Python, plain functions first — no need for a class hierarchy or a library
- matplotlib for the convergence plots
- Write it as a pure function: `run_staircase(question_bank, sub_skill)` -> `MasteryEstimate` so it's trivially testable without any API or database

Week-by-week

- Wk 1-2: dummy question bank + rule-based staircase algorithm working standalone
- Wk 3: swap in Sri Somesh's real graph (Checkpoint 1) once available
- Wk 4-5: mastery scoring model + expose as an API endpoint (Checkpoint 2)
- Wk 6+: convergence study, then help wire the retest loop with Sai Chakrith

Track 2 — Prerequisite Graph & RAG Tutoring

Owner: Sri Somesh S

What this track delivers

- The prerequisite dependency graph (~20-26 sub-skills) tagged against the question bank
- Topological gap ordering so remediation always respects prerequisite order
- The RAG pipeline that generates grounded tutoring content per detected gap
- The Sem 2 generalization to DSA basics (new content only, same logic)

How to build it — suggested approach

Step 1 (Week 1, no dependencies): Finalize the node list from the taxonomy we already drafted, encode it as a plain CSV edge list (sub_skill, prerequisite), load it into `networkx.DiGraph()`, and confirm `nx.is_directed_acyclic_graph()` returns True. This alone satisfies the 'graph design' deliverable and needs nothing from anyone else.

Step 2 — Gap ordering: Given a list of Gap objects (use fake ones until Kaveen's real scoring exists), run `nx.topological_sort()` restricted to the weak sub-skills only, so remediation order always respects dependencies.

Step 3 — RAG corpus: Curate 1-2 pages of concept notes + 3-5 worked examples per sub-skill (you can draft these yourself or adapt from open textbooks/NCERT-level material, citing sources). Chunk into ~300-500 token pieces.

Step 4 — Retrieval + generation: Embed chunks with a free sentence-transformers model (e.g. all-MiniLM-L6-v2) into a lightweight vector store (Chroma or FAISS — both run locally, no paid service needed). For a given gap, retrieve the top 3-4 chunks for that sub-skill, and prompt an LLM (Claude/GPT via API) to write a short explanation *grounded only in those chunks*, plus 2-3 retest questions. This is what makes it 'RAG-grounded' rather than raw LLM generation — cite which chunk each explanation came from.

Step 5 — Generalization test (Sem 2): Once Sem 1 works end-to-end, add the DSA basics content as a second corpus + second edge list CSV. If you built Steps 1-4 generically (subject as a parameter, not hardcoded), this should take days, not weeks — and proves your core claim of subject-agnostic design.

Suggested tools

- NetworkX for the graph
- sentence-transformers (free, local) or OpenAI/Claude embeddings for the vector store
- ChromaDB — easiest to set up locally, good enough for this scale
- Claude or GPT API for the generation step itself

Week-by-week

- Wk 1-2: edge list finalized, loads cleanly into NetworkX, no cycles
- Wk 3: hand off real graph to Kaveen (Checkpoint 1)
- Wk 3-6: corpus curation + embedding pipeline + retrieval working standalone
- Wk 7-8: generation prompts + retest question generation, expose as API (Checkpoint 3)
- Wk 9+ (Sem 2): DSA basics generalization

Track 3 — Data, API & Dashboard

Owner: Sai Chakrith

What this track delivers

- The database schema and API layer that everything else plugs into
- The question bank itself, sourced and tagged
- The React roadmap dashboard the learner actually sees
- End-to-end integration once real modules are ready to swap in

How to build it — suggested approach

Step 1 (Week 1, no dependencies): Design the SQLite schema — one table per contract (questions, sub_skill_graph, mastery_estimates, gaps, tutoring_content, learner_state). Stand up a FastAPI skeleton with endpoints (/diagnostic, /gaps, /tutor, /retest, /roadmap) that return **hardcoded fake JSON** matching the contracts — no real logic yet. This unblocks the dashboard immediately.

Step 2 — Question bank sourcing: Pull or write 15-20 questions per sub-skill (~400-500 total for Sem 1 scope) from standard placement-prep sources (RS Aggarwal, IndiaBix-style banks, past TCS/Infosys/Wipro papers), tag each with sub_skill + difficulty. This is tedious but has zero technical dependency — start immediately and chip away weekly.

UI / Dashboard plan (concrete suggestion)

Since this is the one visual, user-facing piece, here's a specific plan rather than a vague 'build a dashboard':

- **Wireframe first, in Figma or Excalidraw** — 3 screens only: (1) a Start screen, (2) an active Diagnostic screen (question + progress bar), (3) the Roadmap screen. Don't design more than this for the MVP.
- **Roadmap screen is the centerpiece** — use react-flow (a library built exactly for rendering node-and-edge graphs) to literally draw the prerequisite graph, with each node colored by status: gray = not tested, green = mastered, amber = gap detected & in tutoring, red = gap confirmed unresolved. This turns your core research contribution into something visually self-explanatory in a demo.
- **A 'Current focus' card** above or beside the graph — shows the single next action (e.g. 'Review: Compound Interest, then retake a 5-question check') pulled from the topologically-ordered gap list.
- **Keep state management simple** — plain React useState/useReducer is enough at this scale; don't reach for Redux or a heavy state library.
- **Build against the mock API from Step 1 first.** The dashboard should look fully functional weeks before any real diagnostic or RAG logic exists underneath it — this is what makes demos to your supervisor look far more finished than they technically are.

Suggested tools

- FastAPI + SQLite (upgrade to Postgres only if you actually hit concurrency issues — unlikely at this scale)
- React + react-flow for the graph visualization, Recharts for simple progress bars
- Figma or Excalidraw for the 3-screen wireframe before writing any frontend code

Week-by-week

- Wk 1: DB schema + mock API endpoints live
- Wk 1-3: wireframe + dashboard shell built against mocks, in parallel with question bank sourcing
- Wk 3-9: swap mocks for real endpoints as each checkpoint completes
- Wk 10+: full integration, internal testing, UI polish

This week — concrete first actions

Before anyone opens an IDE for their own track, do this together (roughly 2 hours, one sitting):

- Finalize the sub-skill node list and lock the six data contracts as an actual schemas file in the repo
- Create the GitHub repo with three folders: `/diagnostic`, `/knowledge`, `/platform` — one per track, so nobody's work collides
- Agree on the checkpoint dates from the table above and put them on a shared calendar

Then, starting immediately and independently:

- **Kaveen:** dummy question bank + rule-based staircase logic
- **Sri Somesh:** edge list CSV loading cleanly into NetworkX with no cycles
- **Sai Chakrith:** DB schema + mock FastAPI endpoints + Figma wireframe for the 3 screens

Prepared as a working plan for Team AB-05 — Mastrr, Amrita School of AI, Coimbatore.